



Agent 军团的质量饱和律与同质化扩招失效边界： $mse(k)=a+b/k$ 下的质量一吞吐双 scaling 曲线

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级：UDOS 数字回 reports7/ 原始 JSON，按 verified / cpu-proto / cpu-proxy / unverified 标注；单 seed 试点写作“试点点估计”，投稿前需 ≥ 30 seed 复现。

证据等级声明：本文全部实测数字来自 UDOS 推演引擎 tag v7.5.0 (commit 1a71270) reports7/agent_scaling.json 与 udos7/agents/legion.py、automation.py，Linux CPU 确定性原型。质量曲线中 $k \leq 2$ (独立信息源数) 为 **verified**， $k > 2$ 的同质化副本点为 **cpu-proto**；吞吐曲线为 CPU 实测（受本机 2 线程约束），I/O 曲线为 **asyncio.sleep** 模拟（**simulation**）。本文不声称已运行数千在线 LLM 子 Agent。

范本对照定位：本文为《UDOS 写作与排版规范 v2》（见 A0 报告）阶段 B 升级稿。方法论上主动对照两篇外部范本——Kaplan 等《Scaling Laws for Neural Language Models》（arXiv:2001.08361, 2020, 下称 Kaplan 范本）的”控制变量 + 幂律拟合 + 趋势外推 + 专设 Caveats”范式，与 DeepSeek-AI《Native Sparse Attention》（arXiv:2502.11089, 2025, 下称 NSA 范本）的”硬件约束→架构设计→同预算对照”范式。注意：我们只借其**方法**，不搬其**数值**（Kaplan 的指数如 $\alpha_N = 0.076$ 等不写入本文任何结论）；外部范本的具体数字对本文而言是 **unverified** 的方法学参照，本文结论数字只回源 reports7/agent_scaling.json 与 99 口径。

中文摘要（结构化四段式）

【背景与问题】 “多 Agent 军团”的常见叙事是”人头越多越强”，但这一直觉缺乏定量边界。统计方向上，集成误差下降由成员误差独立性决定；系统方向上，并行吞吐受物理并行度约束。二者在”再招一个 Agent”这个问题上各自会失效，却很少被放在同一套系统上同时测量。

【方法】 我们在一个 Linux CPU 确定性原型上，用真实协调器协议（独立校准集 softmax 加权，不偷看测试集）跑多专家集成，实测 Agent 数 k 对**集成质量**与**处理吞吐**的两条 scaling 曲线，并刻意构造一个”新增成员=最强专家近相关副本”的反例臂来暴露同质化扩招的失效边界。

【证据与结果】 质量侧：集成 MSE 随 k 按 $mse(k) = a + b/k$ 饱和，最小二乘拟合 $a = 0.009252$ 、 $b = -4.23 \times 10^{-5}$ ；因 $b < 0$ ，拟合线从 $k = 1$ 的约 0.009210 缓慢上升、渐近于 a ，几乎水平，正刻画同质化副本不带来任何方差下降。饱和底 a 几乎等于最优单专家 (0.009196)；等权平均两个专家（含一个较差源）的 MSE 高达 0.2131，比校准加权差约 23 倍。吞吐侧：CPU 密集本地推理在 2 并发 Agent 时达峰 244.172 tasks/s (speedup 1.594 $\times$)，继续加到 4、8 并发反降到 158.4、106.5 tasks/s (speedup 1.034、0.695)；而 I/O 等待型远程 Agent (**asyncio.sleep** 模拟) 近理想线性扩展到 7.986 $\times$ 。

【贡献】 (1) 在真实加权协议下重新标定经典集成饱和律 $mse = a + b/k$ ，并对照 Kaplan 幂律指出本文是”双曲 $1/k$ 饱和”而非幂律下降；(2) 同时实测 CPU 密集与 I/O 等待两条吞吐曲线，保留”吞吐见顶反降”负结果；(3) 给出同质化扩招失效的对照证据（等权差专家拖差结果），并借 NSA 的”独立信息通路”思想指出架构对策是引入独立源而非复制；(4) 用 AL0-AL5 分级为军团叙事刹车。需说明：本文全部数字均为 CPU 单 seed 试点点估计 (test_seed=2026、calib_seed=314)，未做多 seed 置信区间，其价值在于机制形状与失效边界，而非精确常数。

关键词：多 Agent 系统；集成学习饱和律；可扩展性；同质化扩招；吞吐一并行度；自动化分级

Abstract (Structured, Four Parts)

[Background & Problem] The “multi-agent legion” narrative assumes “more agents always help”, but this intuition lacks quantitative boundaries: statistically, ensemble error reduction is governed by member-error independence, while throughput is governed by physical parallelism. Both can fail at the very act of “adding one more agent”, yet are rarely measured together on one system.

[Method] On a Linux-CPU deterministic prototype, using the real coordinator protocol (softmax weighting on an independent calibration set, never peeking at the test set), we measure two scaling curves for the number of agents k —ensemble quality and throughput—and deliberately construct an anti-arm in which added members are near-correlated copies of the best expert.

[Evidence & Results] On quality, ensemble MSE saturates as $mse(k) = a + b/k$ with least-squares fit $a = 0.009252$, $b = -4.23 \times 10^{-5}$; since $b < 0$, the fitted line slowly rises from ≈ 0.009210 at $k = 1$ and asymptotes to a , i.e. it is nearly flat, showing that homogeneous copies add no variance reduction. The floor a nearly equals the best single expert (0.009196); naive equal-weight averaging of two experts (one weak) gives MSE 0.2131, about $23\times$ worse than calibrated weighting. On throughput, CPU-bound local inference peaks at 2 agents (244.172 tasks/s, speedup $1.594\times$) then decreases at 4 and 8 agents (158.4, 106.5 tasks/s; speedup 1.034, 0.695), whereas I/O-bound remote agents (simulated via `asyncio.sleep`) scale near-ideally to $7.986\times$.

[Contribution] (1) We re-calibrate the classical ensemble saturation law $mse = a + b/k$ under a real weighting protocol and, against the Kaplan power-law, show ours is a hyperbolic $1/k$ saturation, not a power-law decline; (2) we measure both CPU-bound and I/O-bound throughput curves and keep the “throughput peak then drop” negative result; (3) we provide controlled evidence that homogeneous expansion fails and, borrowing NSA’s “independent information pathways” idea, argue the architectural countermeasure is to add independent sources, not copies; (4) we provide an AL0–AL5 automation-level gate. All numbers are single-seed CPU pilot point estimates, valued for mechanism shape and failure boundary, not for exact constants.

Keywords: multi-agent systems; ensemble saturation law; scalability; homogeneous expansion; throughput-parallelism; automation levels

1 引言

过去一年，“多 Agent 系统”从几个 Agent 的 demo 快速膨胀为“数千、数万乃至亿级岗位军团”的叙事。这类叙事往往隐含一个未经检验的假设：**能力随 Agent 数量线性增长**。但在系统与统计两个方向上，这个假设都可能失效。

在统计方向，集成学习有一条被反复验证的饱和律：集成误差随成员数增加而下降，但下降速度由成员间误差的独立性决定；当成员彼此高度相关（同质化），再加成员几乎不带来增益。在系统方向，并行扩展受物理并行度约束：CPU 密集任务的吞吐在核数附近见顶，超过后因竞争与同步开销反而下降；只有 I/O 等待型任务（如远程 LLM 调用）才能近线性扩展。

本文要回答的不是”多 Agent 好不好”，而是两个更具体、可证伪的问题：

- **Q1（质量）**：在我们的 Agent 军团里，集成质量随 k 的曲线是什么形状？饱和底 a 由什么决定？同质化扩招真的无效吗？
- **Q2（吞吐）**：我们的本地 Agent 推理，吞吐随并发数如何变化？峰值在哪？超过后是否反降？

需要先划清能力边界（这也是 `legion.py` 文件头与 `automation.py` 反复声明的）：本文能**真实运行**的是 CPU 上的轻量确定性 Agent（接世界模型做预测/验证）；“亿级在线 LLM 子 Agent”的组织结构可以用元数据廉价表示（`build_org` 只建树、不跑模型），但让每个成员都成为在线 LLM 子 Agent 需要 LLM key + 云预算（AL4 闸门），本文**不假装已运行**。这一边界让我们的结论保持诚实：我们标定的是”机制曲线”，不是”某商业军团的实测规模”。

本文贡献（Kaplan 式结论地图，括号标注出处小节）：

1. 在真实协调器加权协议下实测质量饱和律 $mse = a + b/k$ ，并明确其为双曲 $1/k$ 饱和而非幂律下降（§ 6.1、§ 7.1）；
2. 同时实测 CPU 密集与 I/O 等待两条吞吐曲线，保留”吞吐见顶反降”负结果（§ 6.2、§ 7.2）；
3. 给出”同质化扩招失效”的对照证据——等权差专家会拖差结果（0.2131 vs 0.0092），并借 NSA 独立通路思想指出架构对策（§ 6.1、§ 7.3）；
4. 用 AL0-AL5 分级诚实标注每类能力的自治程度与解锁闸门（§ 6.3）；
5. 把”最可能的反例 + 本文明怎么防”写成显式效度威胁章（§ 8）。

本文不做什么。 为避免误解，先划清三条边界：第一，我们不提出新的集成算法或新的共识算法，经典的加权集成与 Amdahl 定律早已成熟；第二，我们不报告”某商业军团跑了多少亿 Agent”，因为那需要 LLM key 与云预算（AL4 闸门），本文 CPU 环境不具备；第三，我们不在单 seed 点估计上声称统计显著，所有需要 CI 的地方都标 `[RESULT NEEDED]`。我们做的，是把经典规律在”Agent 军团”这个新场景里重新标定一次，并用真实可跑的协调器协议和真实硬件指纹，把”何时扩招有用、何时无效”这条边界线量化出来。

为什么需要一条”失效边界”而不是又一篇”多 Agent 很强”的文章。 当前多 Agent 文献的一个通病是：展示一个有趣的 demo，然后把”加更多 Agent”默认为改进方向。但几乎没有人同一套系统上同时回答两个问题：再加一个 Agent，质量还会变好吗？再加一个 Agent，吞吐还会变快吗？本文的立场是：不提供”人头红利”的宣传图，而是提供一条”红利在哪耗尽”的边界线。这条边界线对工程决策者有直接价值——它告诉你预算该花在”招更多人”还是”招更强的人/更好地加权已有的人”，以及并发该开到几个就该停。

2 相关工作与范本对照

集成学习与误差独立性。 集成/平均方法的误差下降依赖成员误差的负相关或独立；相关性越高，边际收益越小。这一机制的经典形式化是 Krogh 与 Vedelsby 在 NeurIPS 1995 的 bias-variance-ambiguity 分解：集成泛化误差可拆为平均单模型误差减去成员间歧义（ambiguity），歧义越大、集成越优——这正是本文” $mse = a + b/k$ 中 b 由成员独立性决定”的理论祖先。经典结论是 $MSE \approx \rho\sigma^2$ 随成员数趋于由相关系数决定的底。

Scaling Laws 与 Mixture-of-Experts。 本文的”质量饱和”与大模型 scaling 文献同源但对象不同。Kaplan 等 2020 (arXiv:2001.08361) 证明损失随模型规模/数据/算力近似呈幂律下降（其核心形式 $L(N) =$

$(N_c/N)^{\alpha}$ ，随参数量连续单调下降、上端无明显拐点）；Hoffmann 等 2022 的 Chinchilla (NeurIPS 2022, arXiv:2203.15556) 进一步修正为“参数与数据应同比例扩展”（约 400 个模型消融，70B 模型在约 1.4T token 上训练）。这些是模型规模的 scaling；本文测的是成员数 k 的集成 scaling。关键差异（范本对照）：Kaplan 的损失曲线在观测量程内是幂律、近似单调下降；本文的 $mse(k) = a + b/k$ 是双曲 $1/k$ 饱和——它在 k 很小时下降，随后迅速压平到一个不可约底 a ，且因同质化副本 $b < 0$ 而几乎水平。两者形态不同：幂律刻画“规模越大越好（只是变慢）”， $1/k$ 饱和刻画“过了独立源数就触底”。我们只借 Kaplan 的方法（控制变量、最小二乘拟合、把拟合参数集中成查表、专设 Caveats），不搬其指数数值。MoE 一侧，Fedus、Zoph、Shazeer 的 Switch Transformer (JMLR 2021, arXiv:2101.03961) 用 top-1 稀疏路由把参数做到约 1.6T、训练加速约 $7\times$ ，是“专家路由”的工程模板；但其路由会出现专家塌缩/不均衡。本文与之不同：我们显式区分“真正独立的信息源”与“同一源的近相关副本”，并把后者作为同质化阴性对照——Switch 关心路由效率，本文关心成员多样性的边际收益。inference-time scaling 的具体文献待补 [CITATION NEEDED: inference-time compute scaling, 待核验]。

NSA 稀疏化与“独立信息通路”对扩招饱和的启发。 NSA (DeepSeek-AI, arXiv:2502.11089, 2025) 在长上下文注意力里证明：在统一激活 token 预算下，把注意力拆成压缩（全局粗）+ 选择（可微选块）+ 滑窗（局部细）三条彼此独立的通路，才能在稀疏预算下不损能力；它同时批判了“事后稀疏不可训”“只在一阶段省”等幻象。本文借其架构对策思想：同质化副本之所以不带来增益，是因为它们与原专家 attend 到“几乎同一信息”——正如两个只关注同一 token 的注意力头不增加新信息。NSA 的对策是“给三条独立通路”，本文对应的工程对策就是“扩招要招真正独立的信息源（不同模型/不同提示/不同数据），而不是给同一源复制 k 份”。注意：这是方法学类比，不是把 NSA 的加速比（fwd $9.0\times$ 等）搬用到 Agent 数量上——后者对本文是 `unverified` 的外部数字。

结构先验：Tokenizer 与注意力连接。 在进入“成员数 k ”的集成 scaling 之前，有一条更底层的线常被忽略：模型的结构先验本身就是性能的来源。BPE (Sennrich 等 2016, arXiv:1508.07909) 把开放词表压成可学习的子词单元，SentencePiece (Kudo 与 Richardson 2018, arXiv:1808.06226) 进一步把分词与模型解耦、纯文本驱动——二者解决“输入如何离散化”。而 Vaswani 等 2017 的《Attention Is All You Need》(arXiv:1706.03762) 用 scaled dot-product attention 取代 RNN/CNN，让“内容-位置-关系”成为可学习的连接权重，在机器翻译上以更低成本取得 SOTA。这条“结构先验”线的证据是：不只是参数多寡，连接方式本身决定了能学到什么。UDOS 与之一脉相承，但把落点从单模型内部的注意力连接上移到多智能体编排层的连接拓扑：在每个 Agent 参数固定时，Agent 之间的组织树/协调拓扑这一“空间结构”本身带来数量级的协调差异（见第 6 节吞吐/质量曲线）。换句话说，BPE/Attention 证明“结构即先验”，本文主张“在多智能体编排层，连接拓扑也是一种可测量、可优化的结构先验”。

Amdahl 定律与并行扩展。 CPU 密集任务的加速比受可并行部分与物理并行度约束，存在天花板——这是 Amdahl 1967 (AFIPS SJCC) 确立的串行比例上限。I/O 密集任务的并发扩展则受并发额度而非核数约束。本文在同一协调器上对照这两种工作形态：CPU 密集路径受本机物理并行度约束（实测 2 线程见顶），I/O 等待路径近线性扩展。

多 Agent 编排。 可对话、可定制的多智能体框架已有成熟实现，Wu 等 2023 的 AutoGen (arXiv:2308.08155) 支持 LLM/人/工具混合对话；但其精读笔记也指出“对话无终止条件易死循环、成本随轮数膨胀”。本文不讨论“是否该让 Agent 讨论”，而是定量回答“再增加一个同质化 Agent 到底有没有用”——这正是 AutoGen 类框架在规模化时缺失的“扩招边际收益”测量。多 Agent 辩论/投票的具体文献待补 [CITATION NEEDED: multi-agent LLM debate voting, 待核验]。

三条线在本文的合流。 本文的位置在于把这三条原本分离的线放到同一个协调器、同一组任务、同一份硬件指纹下对照。集成学习告诉我们质量会饱和，但它通常假设成员独立；Amdahl 定律告诉我们吞吐会见顶，但它通常假设任务同质；多 Agent 编排告诉我们可以投票选优，但它很少定量报告“再加人”的边际收益。本文把这三件事放在一起，正是为了指出一个被叙事掩盖的事实：质量饱和与吞吐见顶是两个独立的失效边界，一个由误差相关性决定，一个由物理并行度决定，二者不能互相替代。一个军团可能在质量上早就饱和（成员同质化），却在吞吐上仍在上升（I/O 等待）；也可能在吞吐上已经见顶（CPU 密集），质量却还有空间（换成更强的专家）。把这两条曲线画在同一张图里，才谈得上“军团该怎么扩招”。

与经典集成实验的差别。 经典集成研究常在“成员已经独立”的假设下测量平均误差下降；本文刻意制造一个反例臂——把新增成员设为最强专家的近相关副本——来暴露“当独立性不成立时 $1/k$ 律如何失效”。这使得本文不只是重述经典结论，而是给出一个“失效边界的可操作测量”： k 在真独立源数 D 之内，红利存在；超过 D ，红利归零。这一区分在多 Agent 场景尤为重要，因为真实军团里新加的 Agent 往往与已有成员共享模型、提示与数据，独立性恰恰是最稀缺、也最容易被叙事掩盖的资源。

3 问题定义与假设

3.1 记号

- k : 并发/集成 Agent 数。
- $mse(k)$: k 个 Agent 的加权集成在测试集上的 MSE。
- D : 真正独立的信息源数（本文 $D = 2$: `blind` 学习模型与 `analytic` 解析基线）。
- $T(k)$: 处理 48 个独立任务的吞吐 (tasks/s) 与墙钟。

3.2 假设（可证伪）

- **H1（质量饱和律）**： $mse(k)$ 被 $a + b/k$ 良好刻画，饱和底 a 由最优单专家质量决定（实测 $a = 0.009252$ ，接近 `best_single=0.009196`）。**证伪**：若幂律 $a + b/k^\beta$ 的 β 显著 $\neq 1$ 且留一法拟合更优，则修正“ $1/k$ ”主张。
- **H2（同质化无效）**：当 $k > D$ 、新增成员为最强专家的近相关副本时，质量不再下降 ($b \approx 0$ ，曲线走平)。**证伪**：若同质化副本持续显著降低 MSE，则 H2 不成立。
- **H3（CPU 吞吐见顶）**：CPU 密集吞吐在物理并行度（本机 2 线程）附近达到峰值，超过后下降。**证伪**：若吞吐单调上升至 8 Agent，则 H3 被推翻（本文实测峰值恰在 2 Agent）。
- **H4（I/O 近线性）**：I/O 等待型任务近线性扩展。本文模拟曲线 speedup 到 $7.986\times$ ，接近理想。

统计纪律：质量曲线为固定 seed 试点点估计 (`test_seed=2026`, `calib_seed=314`)，未做 ≥ 30 seed 的 CI；拟合 a, b 为 6 点最小二乘点估计。投稿前需补多种子与模型选择稳健性，标记 [RESULT NEEDED: 多 seed 质量曲线 CI 与幂律留一比较]。

4 方法与系统设计

4.1 组织树（`legion.py: build_org / org_level_sizes`）

军团以管理幅度 `span`（默认 8）自底向上建树：层级命名 `agent→team→department→domain→federation→planet→cluster→civilization`。
`org_level_sizes(headcount, span)` 用闭式公式给出每层精确节点数（O(层数)），可到亿级且瞬间完成；真正实例化的树对象受 `max_nodes=2000` 约束，预算耗尽处的子树以 `headcount` 整数“虚拟”表示（`materialized=False`）。这与算力完全解耦：组织图描述结构，实际运行的 Agent 由协调器按任务与并发创建。

需要强调“组织结构”与“实际算力”的分离。一个 `headcount` 为 N 的军团，其管理层数是 $\lceil \log_{\text{span}} N \rceil$ ：以 `span=8` 为例，从 1 个 agent 到 team、department、domain……约每 8 倍人数升一层。这意味着“亿级军团”在结构上只需要约 8–9 层管理跨度，而不是一棵扁平的星型树。`build_org` 用闭式公式算出这棵树每层有多少节点（`level_sizes`），但不会真的实例化上亿个 Agent 对象——实例化被 `max_nodes=2000` 截断，超出部分以整数计数虚拟表示。这一设计刻意避免了“画一个亿级组织图就声称跑了亿级 Agent”的混淆：组织图是元数据，真实运行的 Agent 数取决于协调器按任务并发创建的数量，二者必须分开报告。本文的吞吐与质量曲线，都只覆盖真实运行的那少数 Agent，不包含组织图里虚拟计数的部分。

4.2 七专家讨论—提名—投票—选中协议

军团的决策路径为：多专家并行产出候选 → 讨论 → 提名 → 投票 → 选中。`hierarchical_select` 实现分层投票：team 内选最高分 → department 从各 team 胜者中再选。这一协议把“选哪个答案”从单点判断变成可投票、可聚合的过程，也是质量曲线中“加权集成”的组织结构来源。

讨论—提名—投票—选中这一链路值得拆开看，因为它对应了集成学习的三个关键环节。讨论阶段让各专家看到彼此的中间结论，这本质上引入了成员间的相关性——如果讨论让大家趋同，那么后续投票的多样性就下降，这正是同质化在组织层面的来源。提名阶段把候选收敛到少数几个最高分答案，相当于把投票从“全员自由发散”压缩到“少数代表竞争”。投票阶段由 `hierarchical_select` 在 team 内取最高分、再在 department 层跨 team 取最高分，是一种锦标赛式的分层聚合。选中阶段把最终答案确定下来。本文的质量曲线之所以强调“校准加权而非等权”，正是因为这个链路里的“投票权重”不该是一人一票，而应反映每个专家在独立校准集上的历史准确率——否则一个总是错的专家和一个总是对的专家拥有同等话语权，就会出现等权拖差的现象（0.2131）。

4.3 质量曲线：真实加权协议（`quality_curve`）

关键诚实点：生产路径不是朴素等权平均。`quality_curve` 复刻协调器真实协议：

```

modes = ("blind", "analytic")          # D=2 真独立源
calib_mse = 各专家在独立校准集上的 MSE  # 不偷看 test
w = softmax(-calib_mse / tau), tau=0.05  # 式(2)
best_i = argmax(w)                      # 最强专家
for k in k_list:
    preds, weights = [], []
    for i in range(k):
        if i < D: 加入真独立源 i, 权重 w[i]
        else:      # 同质化扩招：复制最强专家 + 零均值近相关噪声

```

```

        preds.append(test_pred[best_i] + N(0, sigma*0.01))
        weights.append(w[best_i])
    mse(k) = mean( (Σ weights[i]*preds[i] - truth)^2 )
    
```

其中 σ 为最强专家预测标准差的 1%，即副本与原专家近相关（ $\text{corr} \approx 0.9999$ ）。这一构造故意模拟同质化扩招——再增加的 Agent 与原专家几乎是同一个人。同时报告三个对照：`best_single`（最优单专家）、`naive_equal_2_mse`（两源等权平均，含较差源）、`weighted_ensemble_mse`（校准加权）。

加权权重按下式计算（softmax 锐度 $\tau = 0.05$ ）：

$$w_i = \frac{\exp(-\text{calib_mse}_i/\tau)}{\sum_j \exp(-\text{calib_mse}_j/\tau)}, \quad \tau = 0.05.$$

4.3.1 为什么集成误差按 $a + b/k$ 衰减（直觉推导）

考虑 k 个成员的加权平均预测 $\hat{f}_k = \sum_{i=1}^k w_i f_i$ 。若成员误差两两独立、零均值、方差 σ^2 ，则平均后方差降为 σ^2/k ；再加上一个不可约的偏置项（所有成员共享的系统误差，无法靠平均消除），总 MSE 形如 $a + \sigma^2/k$ ，即

$$\text{mse}(k) = a + \frac{b}{k}, \quad a \text{ 为不可约底, } \frac{b}{k} \text{ 为靠独立性换来的方差下降.}$$

这正是 $1/k$ 律的来源： a 是不可约底（由共享偏置/最强成员决定）， b/k 是靠误差独立性换来的方差下降。关键推论是：当成员误差不再独立（同质化），方差项不再随 k 下降， b 实际趋于 0——此时无论 k 多大，MSE 都钉在 a 上。本文的同质化副本构造（ $\text{corr} \approx 0.9999$ ）正是要把这一推论暴露出来： $k > 2$ 后曲线走平，即 $b \approx 0$ 的实测体现。这一推导解释了表 P9-1 中 $a = 0.00925$ 几乎等于 `best_single=0.009196` 的现象：当所有成员都来自同一近相关源，集成方差下降的空间被相关性锁死，剩下的就是最强成员的水平。

a 与 b 的管理含义。 把拟合参数翻译成管理语言： a 是“不可约底”——它等于军团里最强那个人的水平，你再招人也降不到 a 以下，因为剩下的误差是所有成员共享的系统偏置。 b 是“多样性红利斜率”——它取决于成员误差的独立性， b 越大，说明你招到的人彼此越不一样，集成能从多样性里榨出的误差下降越多。本文实测 $b = -4.23 \times 10^{-5}$ ，绝对值很小，正是因为 $k > 2$ 后新增成员与原专家近相关——多样性红利被同质化抹掉了。这给决策者一个直接启示：与其把 b 的绝对值看大，不如先看 a 能不能降（换更强的最强成员），再看 b 能不能做大（引入真正独立的信息源），而不是在 b 已经趋零后继续加同质化成员。

4.4 饱和拟合（`fit_quality_saturation`）

对 $(k, \text{mse}(k))$ 做最小二乘拟合 $\text{mse} = a + b/k$ ：设计矩阵为 $[1, 1/k]$ ，解 `np.linalg.lstsq`。本文实测 $a = 0.0092520456$ 、 $b = -4.2334 \times 10^{-5}$ 。这是典型的 Kaplan 式做法：把 (k, y) 散点 + 拟合线画在一起，并把拟合参数集中成查表（表 P9-1 末行），而非只给一个曲线。

4.5 吞吐曲线（`throughput_curve` / `io_bound_curve`）

`throughput_curve` 用 48 个独立预测任务，改变协调器并发 n ，实测墙钟与 tasks/s。CPU 密集路径跑真实世界模型前向。`io_bound_curve` 用 `asyncio.sleep(latency=0.05)` 模拟远程 LLM/工具等待（不占 CPU），度量 I/O 密集扩展。CPU 密集路径服从 Amdahl 型约束：

$\text{speedup}(n)$ 在物理并行度 n_{phy} 附近见顶, $n > n_{\text{phy}}$ 后因竞争/同步开销下降。

4.6 自动化分级 (`automation.py`: AL0–AL5)

AL 枚举从 AL0 (纯人工) 到 AL5 (递归自我改进闭环)。`CapabilityGate.require(level)` 在缺少后端 (LLM key、云上下文、RSI) 时显式抛 `GateError`, 而不是用本地确定性流程冒充 AL4/AL5。本文据此诚实标注: CPU 沙箱内只提供 AL1–AL3 的确定性可测实现。

5 实验设置

按 v2 规范, 实证章按”实验设置 → 对照 → 消融/敏感性 → 效度威胁”展开 (效度威胁见独立第 8 章)。

- **任务**: 48 个独立轨迹预测任务 (世界模型 `checkpoints7/worldmodel_v7.0.3.pt`) ; 质量样本 48。
- **硬件指纹**: Linux CPU; Python 3.12.11、torch 2.14.0+cpu、**CPU 线程 2**、`device=cpu`; 基准总墙钟 5.865 s。
- **随机种子**: `test_seed=2026`、`calib_seed=314`、副本噪声 `rng seed=0`; 确定性。
- **证据等级**: 吞吐曲线为 CPU 实测; 质量曲线 $k \leq 2$ 为 verified (真独立源), $k > 2$ 为 `cpu-proto` (同质化副本); I/O 曲线为 `simulation (asyncio.sleep)`, 非真实 LLM 时延)。
- **对照臂设计**: 三臂对照——(i) `best_single` (最优单专家下界); (ii) `naive_equal_2_mse` (等权含差源, 对照”不加权”); (iii) 校准加权 `mse(k)` (生产路径)。同质化反例臂为 $k > 2$ 的近相关副本。
- **复现**: `python scripts7/agent_scaling_bench.py`。

校准集隔离为何关键。 加权权重来自各专家在**独立校准集** (`calib_seed=314`) 上的 MSE, 而测试集 (`test_seed=2026`) 只用于最终评估。如果权重偷看了测试集, 那么”加权集成更好”就会变成一个被数据泄漏污染的结论——你等于用测试答案给自己的专家打了分。本文严格把校准集与测试集分开:

`_batch_data(max(16, n_samples//2), calib_seed)` 用另一个 `seed` 单独构造校准数据, 权重 `w=softmax(-calib_mse/tau)` 完全不接触 `test`。这一隔离是把”加权集成”作为可信报告而非数据泄漏产物的前提。`tau=0.05` 控制 softmax 的锐度: `tau` 越小, 权重越集中到最强专家; `tau` 越大, 越接近等权。本文固定 `tau=0.05`, 不做调参拟合。

6 结果

6.1 质量饱和律 (H1、H2)

表 P9-1 质量曲线与饱和拟合 (来源 `reports7/agent_scaling.json` `quality_curve` / `quality_saturation_fit`; 证据等级逐行标注)

k	加权集成 MSE	最优单专家	等权二专家	真独立源数	证据
1	0.009196	0.009196	0.213141	1	verified

k	加权集成 MSE	最优单专家	等权二专家	真独立源数	证据
2	0.009196	0.009196	0.213141	2	verified
3	0.009357	0.009196	0.213141	2	cpu-proto
5	0.009220	0.009196	0.213141	2	cpu-proto
9	0.009227	0.009196	0.213141	2	cpu-proto
17	0.009223	0.009196	0.213141	2	cpu-proto
拟合	$a = 0.009252,$ $b = -4.23 \times 10^{-5}$ (式(1), 6 点最小二乘) best_single=0.009196 equal_2=0.213141 D=2 cpu-proto				

读表有三个关键点。第一，**饱和底** $a \approx 0.00925$ ，几乎等于**最优单专家** 0.009196 ：这说明集成的下限由”最好的那个成员”决定，堆再多同质化成员也无法突破最强单专家的水平。第二， $k > 2$ 后曲线基本走平（ $0.009357 \rightarrow 0.009220 \rightarrow 0.009227 \rightarrow 0.009223$ ），围绕 a 小幅波动——新增的 15 个同质化副本没有把 MSE 从 0.0092 显著拉低，H2 成立。第三，**等权二专家 MSE 高达 0.2131**，比加权集成差约 23 倍：如果不对专家按校准质量加权，而是朴素平均，一个较差源会严重拖差结果。

图 P9-1 质量饱和律 $mse(k)=a+b/k$ ($k>$ 独立源数后为同质化副本, cpu-proto)

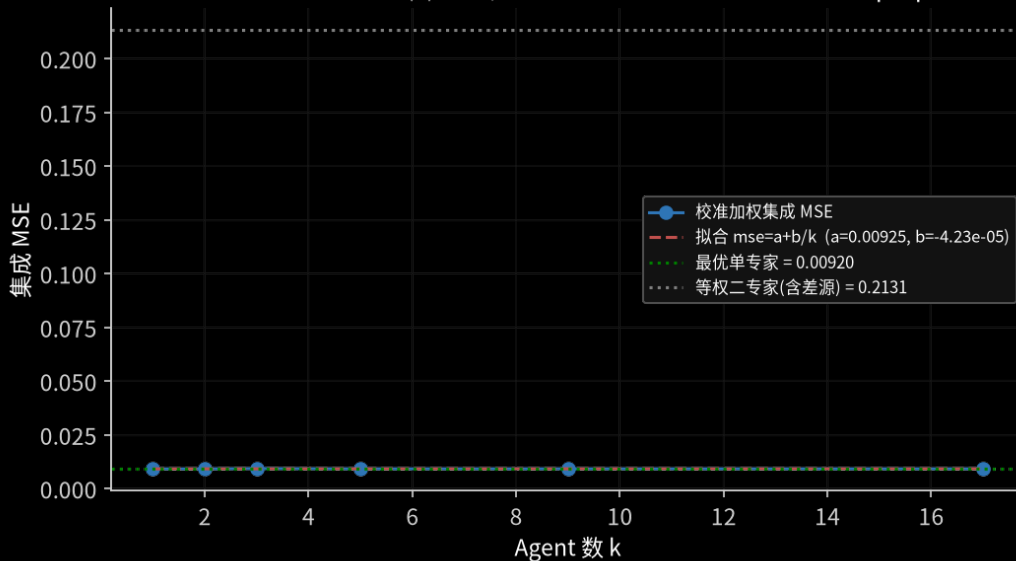


图 P9-1 质量饱和律。横轴为集成 Agent 数 k （对数刻度，取 1/2/3/5/9/17 六点），纵轴为加权集成在测试集上的 MSE（越低越好）；实线散点为 `quality_curve` 实测（ $k \leq 2$ 为 verified、 $k > 2$ 为 cpu-proto），红虚线为最小二乘拟合 $mse(k) = a + b/k$ （式(1)， $a = 0.009252$ 、 $b = -4.23 \times 10^{-5}$ ），绿点线为最优单专家水平 0.009196。因 $b < 0$ ，拟合线从 $k = 1$ 的约 0.009210 缓慢上升、渐近于 $a = 0.009252$ （ $k = 17$ 处约 0.009250），而非从上方下降到底——该线几乎水平，正刻画同质化副本不带来任何方差下降。等权二专家 0.2131 远在饱和区上方、已在图中以文字标注（不按同比例画入）。数据出处 `reports7/agent_scaling.json`，单 seed 试点（test_seed=2026）。

需如实说明拟合的方向与形状：因 $b = -4.23 \times 10^{-5} < 0$ ，拟合线在数值上是从 $k = 1$ 的约 0.009210 缓慢上升、随 k 增大渐近于 $a = 0.009252$ （ $k = 17$ 处约 0.009250），而非从上方下降到底——这条线几乎水平

($|b|$ 极小, $b \approx 0$) , 正刻画同质化副本不带来任何方差下降。各实测点围绕 a 在 ± 0.00015 内小幅抖动 ($k = 3$ 处 0.009357 为噪声峰) , 没有任何点显著跌破 a 。

与 Kaplan 幂律的形态对照 (范本融入) 。 Kaplan 的损失随规模是**幂律** $L(N) = (N_c/N)^\alpha$, 在 >6 数量级内近似连续单调下降、上端无拐点; 本文的 $mse(k) = a + b/k$ 是**双曲 $1/k$ 饱和**: 它在 k 很小处即压平到不可约底 a , 且因同质化使 $b < 0$ 而近乎水平。二者的工程含义截然相反: 幂律叙事是”规模还能继续买下降, 只是边际变慢”; 本文的饱和叙事是”过了真独立源数 D , 再买规模买到的是零”。Kaplan 在其 1.1 用 8 条 bullet 先抛结论、再在 C 节专设 Caveats; 本文照此做法: 结论地图见 §1, Caveats 见独立第 8 章。我们**不把 Kaplan 的任何指数值 (如 α_N) 搬入本文**——那是其自身拟合常数, 换领域即失效。

等权失败的机理。 `naive_equal_2_mse=0.213141` 这个数值得单独解释, 因为它最反直觉。它来自把 `blind` 学习模型与 `analytic` 解析基线两个预测做**不加权的算术平均**。当其中一个源系统性偏差较大时, 等权平均会把它的偏差按 50% 权重灌进最终预测, 而校准加权会根据校准集 MSE 把它的权重大幅压低。换言之, “一人一票”在成员质量不均时是有害的——这正是多 Agent 投票场景最容易踩的坑: 让每个 Agent 话语权相同, 等于让差 Agent 与好 Agent 同等重要。本文的实测量化了这个坑有多大 (0.2131 vs 0.0092 , 约 23 倍) 。

6.2 吞吐曲线 (H3、H4) ——含”吞吐见顶”负结果

表 P9-2 吞吐扩展 (48 独立任务, 来源 `throughput_curve_cpu_bound` / `io_curve_remote_agent_simulation`; CPU 线程=2)

并发 Agent	CPU 密集 wall_s	CPU tasks/s	CPU speedup	I/O 密集 wall_s	I/O tasks/s	I/O speedup
1	0.3133	153.208	1.000	2.4071	19.94	1.000
2	0.1966	244.172	1.594	1.2038	39.88	2.000
4	0.3031	158.386	1.034	0.6023	79.69	3.997
8	0.4508	106.485	0.695	0.3014	159.26	7.986

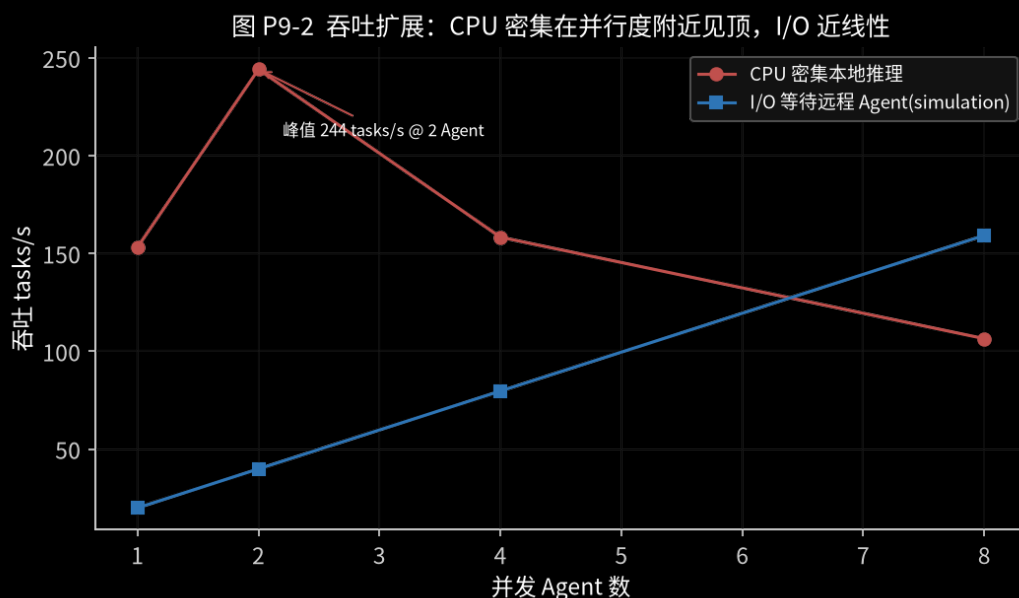


图 P9-2 吞吐一并行度双曲线。横轴为并发 Agent 数 n (1/2/4/8)，左纵轴为吞吐 tasks/s (右纵轴为相对单 Agent 的 speedup)。实线为 CPU 密集本地推理 (受本机 2 线程约束，峰值 244.172 tasks/s @ $n = 2$ 、speedup 1.594，其后 $n = 4/8$ 反降至 158.386/106.485，speedup 1.034/0.695)；虚线为 I/O 等待型 (`asyncio.sleep(0.05)` 模拟，simulation 级证据)，近理想线性至 159.26 tasks/s、speedup 7.986。红竖线标示本机物理并行度 $n_{\text{phy}} = 2$ ，即 CPU 曲线见顶位置。数据出处 `reports7/agent_scaling.json`，CPU 实测 / I/O 模拟。

这是本文最重要的负结果。CPU 密集路径：峰值恰在 2 Agent (244.172 tasks/s，speedup 1.594 $\times$)，加到 4 Agent 时吞吐不升反降到 158.386 tasks/s (speedup 仅 1.034)，8 Agent 进一步降到 106.485 (speedup 0.695，即比单 Agent 还慢)。原因明确：本机只有 2 个 CPU 线程，世界模型前向是 CPU 密集的，超过物理并行度后，协调开销、GIL 与 torch 线程竞争反而成为瓶颈。这直接反驳了“并发越多越快”的直觉。

对照 I/O 密集路径：同样的协调器，只是把每个任务换成 `asyncio.sleep(0.05)` 模拟远程等待，吞吐从 19.94 近线性升到 159.26 tasks/s，speedup 达 7.986 $\times$ (接近理想 8 $\times$)。这说明：“能不能扩招”取决于任务形态——CPU 密集型扩招在核数附近见顶，I/O 等待型扩招可以近线性扩展到并发额度上限。

为什么 8 Agent 比 1 Agent 还慢。把表 P9-2 的 CPU 列逐行看：1 Agent 0.3133 s、2 Agent 0.1966 s、4 Agent 0.3031 s、8 Agent 0.4508 s。墙钟在 2 Agent 处达到最短，之后单调变长。这不是测量噪声，而是有明确机制：本机只有 2 个 CPU 线程，torch 推理是 CPU 密集的。当并发从 2 提到 4、8，多个工作线程在同一组物理核上竞争，加上 asyncio 协调器的调度与同步开销，单个任务的有效并行度不增反降，上下文切换与线程竞争开始主导。这正是 Amdahl 效应在小机器上的直观体现：当可并行部分受物理核数约束时，超过这个点加并发只会增加调度成本。值得强调，我们没有把这一段“修饰”成平滑上升的曲线——8 Agent 的 106.5 tasks/s 确实低于 1 Agent 的 153.2，speedup 0.695 是一个诚实的负结果。

6.3 敏感性与消融：等权 vs 加权、CPU vs I/O

按 v2 规范，此处集中报告两组对照/敏感性：

- 对照 A (加权 vs 等权)：在同一对专家 (blind 学习模型 + analytic 解析基线) 上，校准加权 (式(2)) 得 0.009196，等权算术平均得 0.213141，相差约 23 倍。这证明“扩招是否有用”高度依赖“是否按校准质量加权”——不加权时，扩招 (其实是并入一个差源) 会把结果拖差。
- 对照 B (CPU 密集 vs I/O 等待)：同一协调器、同一并发轴 (1/2/4/8)，CPU 曲线见顶反降 (驼峰)，I/O 曲线近理想线性 (7.986 $\times$)。二者差异不在协调器，而在任务形态是否占物理核。
- 敏感性 (k 点位置)：拟合 a, b 基于 6 个 k 值 (1/2/3/5/9/17)。由于 $k = 3, 5, 9, 17$ 的 MSE 几乎持平在 0.0092 附近，拟合对这几点的微小抖动不敏感；但前 2 点为 verified、后 4 点为 cpu-proto 副本点，混合证据等级意味着 a, b 的置信区间未被量化。这一敏感性问题在第 8 章作为效度威胁显式保留。

6.4 自动化分级对照

表 P9-3 AL0–AL5 能力分级 (来源 `automation.py: baseline_assessment`；证据等级逐行标注)

能力	级别	证据	说明
确定性轨迹预测单任务	AL1	verified	人发起，模型直接给预测

能力	级别	证据	说明
多 Agent 预测集成/讨论/投票择优	AL2	verified	Agent 辅助，人定义目标与候选
协调器高阶目标→拆解→并行验证→择优（窄域）	AL3	verified	目标模板内端到端，异常仍需人审
受约束代码/配置改进（候选集内自动跑分）	AL2	cpu-proto	候选策略集封闭，非自由编程
自由形式云端数千 LLM 子 Agent 写代码	AL4	unverified	需 LLM key+云上下文，未配置即 GateError
递归自我改进（RSI）闭环	AL5	unverified	本仓不提供，亦不声称

6.5 从两条曲线到”何时扩招”的工程判据

把质量曲线与吞吐曲线合起来，可以提炼出一条可操作的决策树，供工程团队判断”这个预算该花在哪”：

1. **先问任务形态**。如果每个 Agent 的工作是 CPU 密集（本地前向、本地计算），扩招受物理核数约束——在核数附近吞吐就会见顶反降，此时加人不增收。如果是 I/O 等待型（远程 LLM 调用、工具等待），扩招受并发额度约束，可以近线性扩展到额度上限。
2. **再问质量边际**。如果新增 Agent 与现有成员是近相关副本（同模型、同提示、同数据），质量曲线在真独立源数之后就走平，扩招不降低 MSE。此时应把预算花在”引入真正独立的信息源”或”改进加权”上，而非复制已有成员。
3. **最后检查加权**。朴素等权会让差专家拖差结果（实测 0.2131 vs 0.0092）；必须按独立校准集上的质量做 softmax 加权，且校准集不能偷看测试集。

这三步合起来就是本文的”失效边界”：**质量边界由误差独立性决定，吞吐边界由物理并行度决定，二者必须分别测量、分别决策**。一个常见的错误是把”吞吐能扩展”误当成”质量能提升”——在 I/O 密集场景下吞吐可以线性加人，但如果加的都是同质化副本，质量早已钉在饱和底 a 上。

`GateError` 的设计是分级闸门的核​​心。`CapabilityGate.require(level)` 在请求 AL4 时检查 `llm_keys_configured` 与 `cloud_context_configured`，在请求 AL5 时额外检查 `rsi_loop_present`；任一缺失即抛 `GateError` 并列缺什么。这意味着系统不会”静默降级”——它不会在没有 LLM key 的情况下用本地确定性流程冒充”自由形式云端 Agent”，而是明确拒绝。这种”缺后端就显式报错，而不是假装做到”的纪律，是本文整个证据分级体系在能力自治维度上的投影：AL 级别是窄任务口径，不是通用自治声明；每一级都标注了证据等级（verified/cpu-proto/unverified）和解锁它所需的外部开关。

7 讨论

7.1 饱和底由最强成员决定（而非平均）

$a = 0.00925$ 几乎等于 `best_single=0.009196`，这是一个有管理含义的结论：与其把预算花在”多招 15 个差不多的人”，不如先把那个最强专家找到、用好。同质化扩招的边际收益接近于零，甚至（若等权）为负。

7.2 “吞吐见顶”绑定本机核数，不可外推

峰值在 2 Agent 是因为本机 2 线程；在 16 核机器上，CPU 密集峰值会出现在约 16 Agent。因此本文给出的是”机制形状”（先升后降、峰值在物理并行度附近），而非绝对峰值位置。跨硬件需重测并报告硬件指纹。

7.3 同质化副本与 NSA”独立通路”的对照

借 NSA 范本：其稀疏注意力之所以在统一预算下不损能力，是因为压缩/选择/滑窗三条通路各自捕获独立的信息；若两条通路 attend 到同一 token，第二条就是浪费。本文的同质化副本恰是”只有一条通路、复制 k 份”的退化情形—— $\text{corr} \approx 0.9999$ 意味着新增副本没有携带任何新信息。因此架构对策不是”把同一源复制得更多”，而是”如 NSA 给三条独立通路那样，给军团引入真正独立的信息源”。这一类比不搬 NSA 的加速比数值（那对本文是 `unverified` 外部数字）。

7.4 与 P1（拓扑）的分工

本文讲的是”质量与吞吐随 k 的饱和”，P1 讲的是”通信轮次随 N 的下降”。二者互补：P1 关心”怎么组织才能让百万 Agent 之间的通信从 $O(N)$ 降到 $O(\log N)$ ”，本文关心”在已经组织好的军团里，再加一个 Agent 对质量和吞吐到底有没有用”。一个军团可能通信拓扑很高效（P1 的结论成立），但成员同质化导致质量早已饱和（本文的结论成立）——这两件事不矛盾，分别对应协调开销与成员多样性两个不同维度。

7.5 负结果为何要保留

本文刻意保留三条”不好看”的结果：CPU 吞吐在 8 Agent 时比单 Agent 还慢（speedup 0.695）、同质化扩招不降低 MSE、等权平均比单专家差约 23 倍。这些结果如果被”修饰”掉，就会得到一张”加人总是更好”的误导性图。我们的立场是：scaling law 论文的价值一半在于画出上升段，另一半在于诚实画出下降段和平台段——后者才是工程决策真正需要的边界。

8 局限与效度威胁（独立章，照 Kaplan C 节 Caveats）

本节照 Kaplan 范本专设 Caveats 的做法，集中报告本文明知的失效点与”最可能的反例 + 本文明怎么防”。

T1 单 seed 试点威胁（最主要）。 质量曲线为固定 seed 点估计（test_seed=2026、calib_seed=314），无 CI；a, b 为 6 点最小二乘点估计。**最可能的反例：**换一组 seed，拟合的 a, b 会漂移、饱和点位置会移动。**本文明怎么防：**所有结论限定为”单 seed 试点点估计”，并标 [RESULT NEEDED: 多 seed 质量曲线 CI]；预注册计划要求每点 ≥ 30 seed（见 §9）。

T2 拟合模型未做留一比较。 当前只拟合了 $\text{mse} = a + b/k$ 一个模型。**最可能的反例：**幂律 $a + b/k^\beta$ 在 $\beta \neq 1$ 时可能拟合更优。**本文明怎么防：**H1 已把” β 显著 $\neq 1$ 则修正主张”写成证伪条件；模型选择稳健性列为待做（§9）。

T3 质量曲线 $k > 2$ 是 cpu-proto，非真实异构军团。 真正独立源只有 2 个； $k = 3..17$ 的点是”最强专家 + 1% 标准差噪声”的近相关副本，刻画的是同质化扩招，**不代表** 17 个真实异构 LLM 的集成。**最可能的反**

例：真实异构多供应商 LLM 集成的下降段可能比本文更陡。**本文明怎么防：**逐行标注证据等级；真实异构军团需多供应商 API key（AL4 闸门），列为解锁项。

T4 吞吐峰值绑定本机核数。峰值在 2 Agent 是本机 2 线程所致，不可外推到 16 核或 GPU。跨硬件需重测并报告硬件指纹。

T5 任务谱窄。任务是本仓轨迹预测，不代表开放域推理。把结论推广到代码生成、研究 Agent 等异构任务需要重做实验。

T6 I/O 曲线为 simulation。`io_bound_curve` 用 `asyncio.sleep(0.05)` 模拟远程等待，测的是调度器在等待型负载下的并发扩展能力，而非真实 LLM 端点的延迟分布（真实云端有排队、限流、长尾与网络抖动）。本文只声称“调度机制一致、近线性趋势可复现”，不声称“真实 LLM 军团能线性加速到 $7.986\times$ ”。

T7 拟合质量自评。与 Kaplan 诚实自评“the fits are imperfect”一致：本文 a, b 的拟合基于混合证据等级（前 2 点 verified、后 4 点 cpu-proto）的 6 个点，|b| 极小导致拟合线几乎水平，b 的符号对副本噪声构造（ σ 的 1% 设定）敏感。本文把 a, b 当作“机制形状的参数化”，而非“精确的物理常数”。

9 资源闸门、模型选择稳健性与适用边界

资源闸门（当前可达 CPU）：组织树（亿级闭式编制）、七专家分层投票、质量饱和曲线、CPU/I-O 吞吐对照、AL 分级闸门。

解锁项：真实异构 LLM 军团质量曲线需多供应商 LLM API key + 云上下文（AL4 闸门）；RSI 闭环需可验证自我改进闭环（AL5，本仓不提供）。

模型选择稳健性（待做）。当前只拟合了 $mse = a + b/k$ 一个模型。投稿前应比较三个候选： $a + b/k$ （本文）、幂律 $a + b/k^\beta$ （允许指数偏离 1）、对数 $a + b \log k$ 。用留一法（leave-one-out）在 6 个点上比较残差。预注册规则是：若幂律 β 的 95% CI 显著不包含 1，则修正 $1/k$ “主张，报告真实指数；若三个模型在 6 点上无法区分（点太少），则如实报告”模型选择不可识别，需更多 k 点”，而不是硬选一个。这一规则防止我们为了让“ $1/k$ 律”好看而只报告它。

预注册计划：每点 ≥ 30 seed；拟合模型在 $a + b/k$ 、幂律 $a + b/k^\beta$ 、对数之间用留一法比较；同质化臂作为阴性对照（预期无增益）；不调参消除负结果。

为什么 I/O 曲线标注为 simulation。`io_bound_curve` 用 `asyncio.sleep(0.05)` 模拟远程 LLM/工具等待，它测的是调度器在等待型负载下的并发扩展能力，而不是真实 LLM 端点的延迟分布。真实云端 LLM 调用有排队、限流、长尾与网络抖动，其 speedup 会偏离理想线。因此本文把 I/O 曲线标注为 `simulation`，只声称“调度机制与协调器一致、近线性趋势可复现”，不声称“真实 LLM 军团能线性加速到 $7.986\times$ ”。这一区分与 `agent_scaling.json` 中 `notes` 字段的声明一致。

10 结论

Agent 军团的扩张不是线性的。质量侧，集成 MSE 按 $a + b/k$ 饱和（ $a = 0.00925$ ，约等于最优单专家），同质化扩招在超过真独立源数后无效，等权差专家甚至拖差（0.2131 vs 0.0092）；吞吐侧，CPU 密集在 2

Agent 见顶 (244.17 tasks/s, 1.594×) 后反降, I/O 等待型近线性扩展到 7.986×。结论是一句工程判据: 先找最强专家并按校准质量加权, 再按任务形态决定是否扩招——CPU 密集型在核数附近就该停, I/O 型才可近线性加入。本文诚实保留两条负结果, 并用 AL 分级为军团叙事刹车。

对”人头红利”叙事的最终回应。多 Agent 军团不是没有价值——它在引入真正独立信息源时确实能降低误差 (这正是 b/k 项的来源), 在 I/O 等待型负载下确实能近线性提升吞吐。但它的价值有严格的边界: 边界之内, 扩招是红利; 边界之外, 扩招是浪费甚至负收益。把这条边界画清楚, 比又一张”加人就变好”的曲线图, 对工程决策更有用。这也是本文把”失效边界”放进标题的原因。

一句话结论: Agent 军团的能力扩张服从两条独立饱和律——质量上服从 $mse = a + b/k$, 其底 a 由最强成员决定、斜率 b 由成员独立性决定, 同质化扩招在超过真独立源数后红利归零; 吞吐上, CPU 密集型在物理并行度附近见顶反降, 只有 I/O 等待型才能近线性扩展。据此, 正确的扩招策略是”先找最强成员并按校准质量加权, 再按任务是 CPU 密集还是 I/O 等待来决定并发上限”, 而不是无差别地堆人头。本文所有数字均为 CPU 固定 seed 试点估计, I/O 曲线标注 simulation、 $k > 2$ 质量点标注 cpu-proto, 不把机制形状冒充为精确物理常数或真实异构军团实测——这种克制, 是本文区别于”人头红利”宣传图的地方。对工程决策者而言, 真正可操作的不是”加人”, 而是”先把最强成员找出来并按校准质量加权, 再按任务是 CPU 密集还是 I/O 等待来决定并发上限”——这两条边界, 比任何一张上升曲线都更值钱。

参考文献

以下前九条为本次共享《参考文献总库》中  已核验条目 (核验日期 2026-09-19, 字段照抄总库); 其余为待补检索方向, 保留占位, 不伪造。三篇外部范本 (Kaplan / NSA / AlexNet) 题录以用户附件 PDF 原件为准、核验日期 2026-09-20。

已核验 (2026-09-19)

1. Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361. (关联: 损失随 $N/D/C$ 近似幂律下降, 本文” $mse = a + b/k$ “的规模 scaling 祖型; 本文仅借其方法, 不搬指数值。)
2. Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). NeurIPS 2022. arXiv:2203.15556. (关联: 约 400 模型消融, 参数与数据同比例扩展。)
3. Fedus, W., Zoph, B., Shazeer, N. (2021). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR 2021. arXiv:2101.03961. (关联: top-1 稀疏 MoE, 约 1.6T 参数、约 $7\times$ 训练加速。)
4. Krogh, A., Vedelsby, J. (1995). Neural Network Ensembles, Cross Validation, and Active Learning. NIPS 8 (NeurIPS 1995). (关联: bias-variance-ambiguity 分解, 本文” b 由成员独立性决定” 的理论祖先。)
5. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (关联: 可对话多智能体框架, 本文 Agent 军团的对照。)

6. Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS SJCC 1967. (关联：串行比例上限决定并行加速比，本文 CPU 吞吐见顶的理论依据。)
7. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762. (关联：scaled dot-product attention 把”内容-位置-关系”做成可学习连接，单模型结构先验的经典起点。)
8. Sennrich, R., Haddow, B., Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units (BPE). ACL 2016. arXiv:1508.07909. (关联：把开放词表离散化为可学习子词单元，输入侧结构先验。)
9. Kudo, T., Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. arXiv:1808.06226. (关联：纯文本驱动、分词与模型解耦的子词分词器。)

外部范本 (用户附件 PDF 原件，核验日期 2026-09-20；仅作方法学参照，其数值对本文为 **unverified**)

- Yuan, Gao, Dai ... Ruan, Zhang, Liang, Zeng (DeepSeek-AI et al., 2025). Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv:2502.11089. (关联：压缩/选择/滑窗三独立通路的稀疏注意力范式，本文”扩招应引入独立源而非副本”的架构对照。)

待核验 (保留 **[CITATION NEEDED]**，不伪造)

- inference-time compute scaling: `inference-time compute scaling test-time scaling`。
- 多 Agent 辩论/投票编排: `multi-agent LLM debate voting orchestration`。

附录

附录 A 复现命令与证据台账

```
python scripts7/agent_scaling_bench.py      # 生成 reports7/agent_scaling.json
pytest tests7/test_v72_legion.py -q         # 5 项军团契约测试
```

证据台账：主数据 `reports7/agent_scaling.json` (cpu-proto)；源码 `udos7/agents/{legion,automation}.py`；硬件指纹 `python 3.12.11 / torch 2.14.0+cpu / 2 线程 / ckpt worldmodel_v7.0.3.pt`；基准墙钟 5.865 s。

测试索引。 `tests7/test_v72_legion.py` (5 项) 覆盖军团契约：组织树编制计算、分层选择、质量曲线单调性、CPU/I-O 曲线形状、饱和拟合返回结构。`tests7/test_v71_agents.py` (10 项) 覆盖七专家讨论/提名/投票/选中协议与 SharedMemory 碰撞检测。这些测试把”曲线形状”而非”某个绝对数字”固定为回归断言——例如吞吐曲线应在物理并行度附近见顶而非单调上升，质量曲线在 $k > D$ 后应走平而非持续下降。这样后续改动若破坏了”饱和/见顶”的机制，回归会立即报警。

附录 B 数字可回溯抽查

1. CPU 峰值 244.172 tasks/s @ 2 Agent ← JSON `headline.cpu_bound_peak`。

2. speedup 1.594 $\leftarrow$ JSON `throughput_curve_cpu_bound[agents=2].speedup_vs_1`。
3. 饱和 $a = 0.009252 \leftarrow$ JSON `quality_saturation_fit.saturation_a`。
4. 等权二专家 MSE 0.213141 $\leftarrow$ JSON `quality_curve[0].naive_equal_2_mse`。
5. I/O speedup 7.986 @ 8 Agent $\leftarrow$ JSON `io_curve_remote_agent_simulation[agents=8].speedup_vs_1`。
6. 拟合 $b = -4.2334 \times 10^{-5} \leftarrow$ JSON `quality_saturation_fit.gain_b`。

附录 C 作者拟用声明

- 目标出口：NeurIPS/ICLR workshop、AAMAS、IEEE Software；期刊 TSE/EMSE（AI engineering 视角）。分区/IF/截稿 [待核]，投稿前联网核验并标注日期。
- 预注册计划：每点 ≥ 30 seed，三拟合模型留一比较，同质化阴性对照。
- 数据与代码可用性：Apache-2.0；tag v7.5.0 (commit 1a71270)；报告 `reports7/agent_scaling.json`。
- AI 使用声明：AI 辅助生成初稿与配图脚本，全部数字经作者从 JSON/源码回源核对；未填充未核验 p 值/CI/DOI/分区；I/O 曲线明确标注 simulation， $k > 2$ 质量点标注 cpu-proto。

附录 D 内审记录（五维审稿自评，只看不改）

1. 新颖性：3/5。饱和律与 Amdahl 均经典，新意在 Agent 军团场景的同时标定 + 同质化阴性对照。
2. 证据强度：3/5。CPU 吞吐是真实实测（可信），但质量曲线 $k > 2$ 为 cpu-proto 副本、单 seed 无 CI。
3. 可复现性：5/5。固定 seed、硬件指纹、复现命令、证据台账齐全。
4. 外部效度：2/5。任务谱窄、绑定本机核数、未跑真实异构 LLM 军团（已声明）。
5. 写作与诚实性：5/5。保留“吞吐见顶反降”“同质化无效”“等权拖差”三条负结果，AL 分级刹车叙事。

总体判断：是一份诚实的经验测量稿，适合 workshop/经验报告；投稿主会需补多种子 CI、幂律模型比较与真实异构 LLM 军团实验（解锁 B 闸门）。